SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **KANMANI K B (192524306)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. **Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty**
ii. **Explain how A* would handle the same situation, including how it balances cost and heuristic**
iii. **Analyze the impact of heuristic accuracy on both algorithms**
iv. **Discuss how dynamic changes (blocked paths) affect search performance**
v. **Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety**

SOLUTION:

1. Greedy Best-First Search in the Scenario

Greedy Best-First Search is an informed search algorithm that selects the next node based only on the heuristic value \(h(n)\). In this scenario, the drone uses straight-line distance to estimate how close each location is to the destination.

Working:

1. The drone identifies possible routes to the destination.
2. It calculates the straight-line distance from each possible location to the destination.
3. It selects the location with the lowest heuristic value.
4. It continues until it reaches the destination or encounters a blocked path.

Strengths:

- Finds promising routes quickly.
- Requires less computation in many situations.
- Useful when emergency medicines must be delivered urgently.

Weaknesses:

- Ignores the actual cost already incurred.

- May select a short route with dangerous weather or blocked paths.
- Does not guarantee the shortest or safest route.

2. How A* Handles the Same Situation

A* Search considers both the actual cost and the estimated remaining cost. It uses the evaluation function:

\[f(n)=g(n)+h(n) \]

Where:

- \(g(n)\): Actual travel cost from the starting point to node \(n\), including known distance, time, or risk costs.
- \(h(n)\): Estimated remaining cost to reach the destination.
- \(f(n)\): Estimated total cost of the route through node \(n\).

The drone selects the node with the lowest \(f(n)\). For example, a longer route with clear weather and no obstacles may be preferred over a shorter route with high risk.

A* can find an optimal route if the cost model is accurate and the heuristic is admissible, with suitable search conditions.

3. Impact of Heuristic Accuracy

- Accurate heuristic: Both algorithms can identify promising routes more effectively.
- Inaccurate heuristic: Greedy Best-First Search may choose a poor route because it relies entirely on the heuristic.
- A Search:* It can use the actual cost already incurred to reduce the effect of a misleading heuristic.
- Overestimated heuristic: A* may lose its guarantee of optimality if the heuristic overestimates the remaining cost.

4. Effect of Dynamic Changes

Blocked roads, sudden storms, and changing terrain costs can make previously calculated routes invalid.

- Greedy Best-First Search may quickly select another route, but it can repeatedly make locally attractive choices.
- A* may need to update costs and search again when the environment changes.
- Replanning algorithms, such as D* Lite, can efficiently update routes when new information becomes available.

5. Recommended Algorithm

A with dynamic replanning is the most suitable approach.*

It balances actual travel cost and estimated remaining cost, helping the drone find efficient routes while considering weather and terrain. Dynamic replanning allows it to respond to newly blocked paths. Safety constraints, such as avoiding dangerous weather and restricted areas, must also be enforced.

Greedy Best-First Search is faster in some situations, but A* provides a better balance between route quality, safety, and efficiency when sufficient computation time is available.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. **Explain how Hill Climbing would work in this scenario and identify its limitations**
ii. **Discuss issues like local maxima, plateaus, and ridges with real-world interpretation**
iii. **Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations**
iv. **Recommend the best approach and justify your choice based on scalability and adaptability**

SOLUTION:

Formulation as an Optimization Problem

The objective is to find signal timings that minimize waiting time, fuel consumption, and traffic congestion across hundreds of intersections.

Let the objective function be:

$$\min J=w_1W+w_2F+w_3C$$

Where:

- W: Total vehicle waiting time.
- F: Total fuel consumption.
- C: Traffic congestion measure.
- w_1,w_2,w_3: Weights representing the importance of each objective.

Decision variables:

- Green-light duration for each signal.
- Signal phase sequence.
- Offset between neighboring traffic signals.

Constraints:

- Minimum and maximum green-light durations.
- Safe transition periods between signals.
- Pedestrian crossing requirements.
- Traffic flow and road-capacity limitations.

Traditional exhaustive search is inefficient because the number of possible signal combinations grows rapidly with the number of intersections. It also struggles to react quickly to continuously changing traffic.

1. Hill Climbing in Traffic Optimization

Hill Climbing is a local search algorithm that repeatedly improves the current solution.

Working:

1. Generate an initial set of traffic signal timings.
2. Evaluate total waiting time, fuel consumption, and congestion.
3. Change one or more signal timings slightly.
4. Compare the new solution with the current solution.
5. Keep the improvement and repeat until no better neighboring solution is found.

For example, if a road has a long queue, the algorithm may increase its green-light duration and reduce unnecessary waiting.

Advantages:

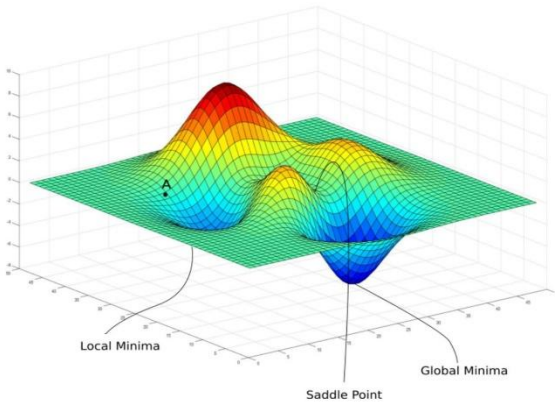
- Simple to implement.
- Requires relatively little memory.
- Can produce quick improvements.
- Suitable for repeated local adjustments.

Limitations:

- May become trapped in a local optimum.
- Does not guarantee the globally best solution.
- Can struggle with large, interconnected traffic networks.

2. Local Maxima, Plateaus, and Ridges

In a minimization problem, the corresponding difficulty is often described using local minima rather than local maxima.

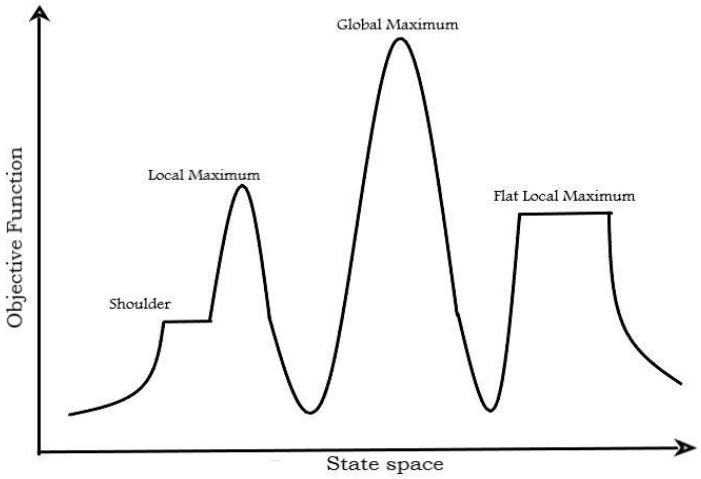


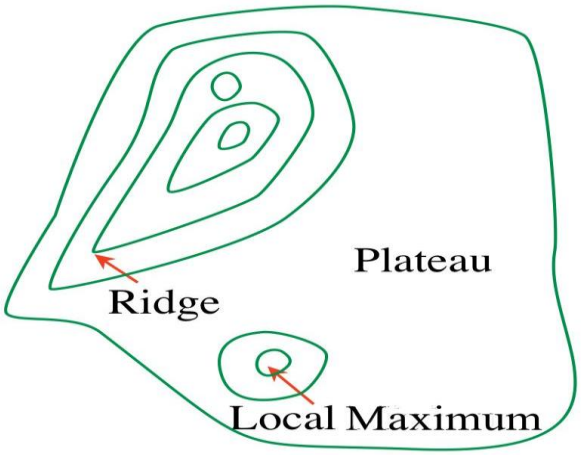
Local optimum

A signal configuration improves traffic in one area but performs poorly across the entire city. The algorithm stops because nearby changes appear worse.

Plateau

Several timing configurations have almost identical performance. Hill Climbing cannot easily identify which direction will improve traffic.





Ridge

Improving traffic may require coordinated changes at several intersections. Changing only one signal at a time may fail to discover the better configuration.

3. How Simulated Annealing and Genetic Algorithms Help

Simulated Annealing (SA)

- Starts with an initial solution.
- Tries neighboring timing configurations.
- Accepts better solutions.
- Sometimes accepts worse solutions early in the search to escape local optima.
- Gradually reduces the probability of accepting worse solutions.

This allows the algorithm to explore alternatives instead of getting stuck immediately.

Genetic Algorithm (GA)

- Creates a population of possible signal-timing configurations.
- Evaluates each configuration using a fitness function.
- Selects better solutions for reproduction.
- Applies crossover and mutation to create new solutions.
- Repeats the process over multiple generations.

GA explores several candidate solutions simultaneously, making it useful for complex networks with many interacting intersections.

4. Recommended Approach

A Genetic Algorithm combined with real-time traffic feedback is a suitable choice for large smart cities.

It can explore multiple timing configurations and handle complex, interconnected objectives. The system can periodically update its population using fresh traffic data.

For strict real-time requirements, a practical design is to use GA for broader optimization and a faster local controller for immediate signal adjustments. Simulated Annealing is also useful when a simpler single-solution search is preferred.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. **Explain why this problem requires an online search agent instead of offline search**
ii. **Analyze the challenges of partial observability and dynamic environments**
iii. **Describe how the rover builds and updates its internal model**
iv. **Compare online search with uninformed and informed search strategies**
v. **Justify the most suitable approach considering uncertainty, adaptability, and safety**

SOLUTION:

1. Why Online Search Is Required

An offline search agent plans using a complete map before beginning its journey. However, a Mars rover does not initially know every obstacle, crater, or terrain condition.

An online search agent selects an action, observes the environment, updates its knowledge, and then chooses its next action.

For example, if the rover discovers a crater blocking its planned path, it can update its map and select an alternative route.

Online search is therefore necessary because the rover must make decisions while exploring an unknown environment.

2. Challenges of Partial Observability and Dynamic Environments

Partial observability:

- Sensors provide only limited information.
- Obstacles may be hidden or outside the sensor range.
- Terrain conditions may be uncertain.
- The rover cannot rely on a complete map.

Dynamic conditions:

- Dust, terrain instability, and equipment conditions may change.
- Communication delays can prevent immediate instructions from Earth.
- New observations may make an earlier route unsafe.

The rover must make decisions using incomplete information and continuously assess the safety of its path.

3. Building and Updating the Internal Model

The rover maintains an internal representation of the environment using sensor readings and observations.

The process is:

1. Sense: Collect camera, distance, and terrain data.
2. Map: Represent known areas as free, occupied, or unknown.
3. Update: Modify the map when new obstacles are discovered.
4. Plan: Select a path toward a sample collection point.
5. Act: Move a safe distance along the selected route.
6. Replan: Change the route if a hazard is detected.

This repeated process allows the rover to explore safely without requiring complete prior knowledge.

4. Comparison of Search Strategies

Strategy	**Description**	**Suitability**
Uninformed search	Explores without heuristic guidance, such as BFS or DFS.	Useful for simple problems but may explore many unnecessary locations.
Informed search	Uses heuristic information, such as A* using distance to the target.	More efficient when useful map and cost estimates are available.
Online search	Acts, observes, learns, and replans during exploration.	Essential when the environment is initially unknown.

Online search is not itself a replacement for informed or uninformed search. An online rover can use an informed method such as A* on the currently known map.

5. Recommended Approach

Online informed search with incremental replanning is the most suitable approach.

The rover should combine sensor-based mapping, heuristic path planning, and rapid obstacle avoidance. Algorithms such as D* Lite can help update routes when the map changes.

Safety should take priority over speed. The rover should avoid uncertain or hazardous terrain, maintain a safe fallback action, and replan whenever new information makes its current route unsafe.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. **Clearly define variables, domains, and constraints with explanation**
ii. **Explain how backtracking search works in this scenario**
iii. **Discuss how constraint propagation (e.g., forward checking) improves efficiency**
iv. **Analyze real-world challenges and justify the best strategy for scalability**

SOLUTION:

1. Formulation as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints. The objective is to assign values to all variables while satisfying every required constraint.

Variables

Let each course be a variable:

$$X = \{E_1, E_2, \dots, E_n\}$$

Each variable represents the scheduled time slot and, where necessary, the examination hall for a course.

Domains

The domain of each course contains all permitted time-slot and hall combinations.

For example:

$$D(E_1) = \{(S_1, H_1), (S_1, H_2), (S_2, H_1)\}$$

Some combinations may be removed because of hall capacity, accessibility, or faculty availability.

Constraints

- No student clashes: Two exams taken by the same student cannot occur at the same time.
- Hall capacity: A hall must have enough seats for the assigned students.
- Hall availability: A hall cannot host two exams simultaneously.
- Ordering constraints: Certain subjects must occur before other subjects.
- Faculty availability: Exams requiring a particular faculty member must be scheduled when that person is available.

- Special accommodations: Students requiring accessible halls or additional time must receive appropriate arrangements.

The objective is to find a valid assignment satisfying all hard constraints. Soft constraints, such as avoiding too many exams on consecutive days, can be optimized afterward.

2. How Backtracking Search Works

Backtracking assigns values to variables one at a time and reverses assignments when a constraint is violated.

Steps:

1. Select an unscheduled course.
2. Assign a valid time slot and examination hall.
3. Check whether the assignment satisfies all constraints.
4. If valid, continue with the next course.
5. If no valid assignment is possible, undo the previous assignment and try another value.
6. Stop when all courses are assigned successfully or all possibilities have been exhausted.

Example: If Mathematics and Physics share students, they cannot be scheduled in the same time slot. If the algorithm assigns both exams to the same slot, it must backtrack and move one of them.

3. How Constraint Propagation Improves Efficiency

Constraint propagation removes impossible values from domains before they are selected.

Forward checking: After assigning a course to a time slot, the algorithm removes conflicting slots from the domains of related courses.

For example, if Mathematics is scheduled in Slot 1, Slot 1 is removed from the available domain of Physics when they share students.

Benefits include:

- Detecting conflicts early.
- Reducing the number of assignments explored.
- Avoiding unnecessary backtracking.
- Improving overall scheduling efficiency.

Other useful methods include arc consistency, which checks whether domain values remain compatible across related variables.

4. Real-World Challenges and Scalable Strategy

Large universities may have thousands of students, hundreds of courses, limited halls, and complicated faculty schedules. The number of possible assignments can grow exponentially.

A scalable solution should combine:

- Minimum Remaining Values (MRV): Select the course with the fewest legal assignments first.
- Degree heuristic: Prefer courses that constrain the largest number of other courses.
- Least Constraining Value (LCV): Choose the assignment that leaves the most options for remaining courses.
- Forward checking or arc consistency: Eliminate conflicts early.
- Optimization methods: Improve preferences after satisfying hard constraints.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. **Explain how the Minimax algorithm models this problem**
ii. **Analyze the computational challenges of large game trees**
iii. **Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**
iv. **Design an evaluation function and justify the factors considered**
v. **Recommend a strategy for balancing optimality and real-time performance**

SOLUTION:

1. How Minimax Models the Problem

Minimax is a decision-making algorithm for competitive games in which one player tries to maximize the score while the opponent tries to minimize it.

The game is represented as a tree:

- MAX nodes: AI's turn; select the highest-valued move.
- MIN nodes: Opponent's turn; assume the opponent selects the move that gives the AI the lowest value.
- Leaf nodes: Game outcomes or states evaluated by a scoring function.

The algorithm evaluates possible moves and chooses the move that provides the best outcome against an optimal opponent.

For example, an AI may choose a defensive move rather than an aggressive one if the aggressive move allows the opponent to win on the next turn.

2. Computational Challenges of Large Game Trees

If each state has b possible moves and the search explores depth \(d\), the number of nodes is approximately:

$$O(b^d)$$

For example, with a branching factor of 10 and a search depth of 6, a complete tree may contain around one million leaf nodes.

This creates several problems:

- High computation time.
- Large memory requirements if the entire tree is stored.

- Difficulty meeting real-time response deadlines.
- Incomplete information about distant future outcomes.

Therefore, searching every possible move is often impractical in real-time strategy games.

3. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning improves Minimax by skipping branches that cannot change the final decision.

It maintains two values:

- \(\alpha\): The best value MAX can guarantee so far.
- \(\beta\): The best value MIN can guarantee so far.

Pruning occurs when:

$$\alpha \geq \beta$$

Example:

Suppose MAX has already found a move worth 8 points. While evaluating another move, the opponent can force an outcome worth at most 5 points. MAX will never choose that second move over the known value of 8, so the remaining branches of that move can be skipped.

Alpha-Beta Pruning produces the same decision as Minimax when both search the same tree and use the same evaluation function, but it can examine far fewer nodes.

With good move ordering, its best-case time complexity is approximately:

$$O\left(b^{d/2}\right)$$

Its worst-case complexity remains $O(b^d)$

4. Designing an Evaluation Function

When the search reaches a depth limit, an evaluation function estimates how favorable a state is for the AI.

A possible evaluation function is:

$$Eval(s) = w_1M + w_2R + w_3T \\ +w_4O + w_5P - w_6C$$

Where:

- \(M\): Military strength.
- \(R\): Available resources.
- \(T\): Territory controlled.
- \(O\): Objective or mission progress.
- \(P\): Positional advantage.
- \(C\): Expected damage or vulnerability.
- $w_1, \dots, w_6$Weights representing the importance of each factor.

The weights should reflect the game's objectives. For example, a strategy game focused on territory may assign more weight to territorial control, while a resource-management game may prioritize resources and economic growth.

The function should also assign very high positive or negative values to winning and losing states.

5. Recommended Strategy for Real-Time Performance

Use iterative-deepening Minimax with Alpha-Beta Pruning.

The AI searches to increasing depths while time permits. It stores the best move found at each completed depth and can return that move if the time limit is reached.

Additional improvements include:

- Move ordering: Examine promising moves first to increase pruning.
- Transposition tables: Reuse evaluations of previously visited states.
- Evaluation functions: Estimate states that cannot be searched to the end.
- Time limits: Stop searching before the response deadline.
- Selective search: Spend more computation on strategically important positions.

This approach balances decision quality, computational efficiency, and real-time responsiveness. Although it may not always find the globally optimal move when search is cut off, it can make strong decisions within strict time limits.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand